This guide breaks down every program from **Experiment 1 to Experiment 10** in this repository. It is tailored for writing these programs, explaining the logic, and answering viva/theory questions in an exam.

Table of Contents

1. Exp 1: 8086 Basics — "Hello World" & Addition
2. Exp 2: 8086 Memory Block Transfer (Within & Across Segments)
3. Exp 3: 8086 String Operations (Length, Compare, Palindrome, Reverse)
4. Exp 4: Booth's Multiplication Algorithm
5. Exp 5: Division Algorithms (Restoring & Non-Restoring)
6. Exp 6: Amdahl's Law (Speedup & Efficiency)
7. Exp 7: Page Replacement Policies (FIFO & LRU)
8. Exp 8: Cache Memory Direct Mapping
9. Exp 10: Pipelining Simulation & Hazard Stalls

Experiment 1: 8086 Basics

Common 8086 Structure to Remember for the Exam

Every standard MASM/TASM 8086 program follows this skeleton:

```
.MODEL SMALL          ; Allocates memory model (code <= 64KB, data <= 64KB)
.STACK 100H           ; Reserves 256 bytes (100h) for the stack
.DATA                 ; Start of data segment (variables declared here)
    ; variables
.CODE                 ; Start of code segment (instructions written here)
MAIN PROC
    MOV AX, @DATA     ; 8086 cannot load @DATA directly into DS
    MOV DS, AX        ; Initialize Data Segment register
    ; your logic
    MOV AH, 4CH       ; DOS interrupt function 4CH = exit/terminate process
    INT 21H
MAIN ENDP
END MAIN
```

Program 1.1: Display "Hello World" (`HELLO.ASM`)

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG DB 'Hello World$'       ; '$' is the string terminator required by
```

```
DOS function 09H

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX                  ; Initialize DS

    LEA DX, MSG                 ; Load Effective Address of MSG into DX
    MOV AH, 09H                 ; DOS function 09H: Display string starting
at DS:DX
    INT 21H                     ; Call DOS interrupt

    MOV AH, 4CH                 ; DOS function 4CH: Exit program
    INT 21H
MAIN ENDP
END MAIN
```

- **How it works**:
  - `MSG DB 'Hello World$'`: Defines bytes (`DB`). In DOS, interrupt `21H` with `AH = 09H` prints characters until it encounters the `$` character.
  - `LEA DX, MSG`: Loads the offset address of `MSG` into `DX`.
  - `INT 21H`: Triggers DOS service routines.

Program 1.2: Add Two Numbers (`ADD2NUMS.ASM`)

16-Bit Addition (As per syllabus):

```
.MODEL SMALL
.STACK 100H

.DATA
    NUM1 DW 1234H     ; DW = Define Word (16 bits / 2 bytes)
    NUM2 DW 5678H
    SUM  DW ?         ; '?' reserves uninitialized memory

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM1      ; Load 16-bit NUM1 into accumulator AX
    ADD AX, NUM2      ; AX = AX + NUM2
    MOV SUM, AX       ; Store result in memory location SUM

    MOV AH, 4CH       ; Exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

(Note: If asked for 8-bit addition, replace `DW` with `DB` and `AX` with `AL`.)

Experiment 2: Memory Block Transfer

Key Instructions to Know:

- `MOVSW` (Move String Word): Copies 1 word (16 bits) from source address `DS:SI` to destination address `ES:DI`. After copying, it automatically increments `SI` and `DI` by 2 (if Direction Flag `DF = 0`).
- `CLD` (Clear Direction Flag): Ensures `DF = 0` (forward scanning / auto-increment).
- `REP`: Repeats the string instruction `CX` times, decrementing `CX` after each transfer until `CX = 0`.

Program 2.1: Transfer 10 Words within the Same Segment (`1000:0000` to `1000:2000`) (`COPYINSEG.ASM`)

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AX, 1000H
    MOV DS, AX          ; Source Segment DS = 1000H
    MOV ES, AX          ; Destination Segment ES = 1000H

    MOV SI, 0000H       ; Source Offset SI = 0000H
    MOV DI, 2000H       ; Destination Offset DI = 2000H
    MOV CX, 10          ; Counter = 10 words (each word is 16 bits)

    CLD                 ; Clear Direction Flag (auto-increment SI & DI)
    REP MOVSW           ; Repeat copy word CX times

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

`CLD` clears the Direction Flag, so string instructions move forward: `SI` and `DI` increase after each word. `REP MOVSW` repeats `MOVSW` until `CX` becomes zero. Each repetition copies one 16-bit word from `DS:SI` to `ES:DI`, so `MOV CX, 10` copies 10 words.

Program 2.2: Transfer 10 Words Across Segments (`1000:0000` to `2000:0000`) (`COPYXSEG.ASM`)

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
```

```
    MOV AX, 1000H
    MOV DS, AX          ; Source Segment DS = 1000H

    MOV AX, 2000H
    MOV ES, AX          ; Destination Segment ES = 2000H

    XOR SI, SI          ; SI = 0000H (XORing register with itself clears it
to 0)
    XOR DI, DI          ; DI = 0000H
    MOV CX, 10          ; Count = 10 words

    CLD
    REP MOVSW           ; Copies from 1000:0000 to 2000:0000

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Experiment 3: String Operations

Program 3.1: Length of a String (`STRLEN.ASM`)

- **Concept**: Use `LODSB` (loads byte at `DS:SI` into `AL` and increments `SI`). Loop until character matches `'$'`.

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB "Hello World$"
    LEN  DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1        ; SI points to start of STR1
    XOR CX, CX          ; CX = 0 (Length counter)

L1:
    LODSB               ; AL = [SI], SI = SI + 1
    CMP AL, '$'         ; Check for end of string
    JE DONE             ; If equal to '$', exit loop
    INC CX              ; Otherwise, increment counter
    JMP L1

DONE:
    MOV LEN, CL         ; Store length in variable LEN
```

```
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

`XOR CX, CX` sets `CX` to zero by XORing the register with itself. This is a common and efficient way to initialize the length counter before the loop. Each non-`$` character increases `CX` by one, and the final value is stored as the string length.

Program 3.2: Check if Two Strings are Equal (`CMPSTR.ASM`)

- **Concept**: Use `CMPSB` (compares byte at `DS:SI` with byte at `ES:DI`) combined with prefix `REPE` (Repeat while Equal).

```
.MODEL SMALL
.STACK 100H
.DATA
    STRING1  DB 'HELLO$'
    LEN1     EQU $ - STRING1 - 1    ; Assembler computes length excluding
'$'
    STRING2  DB 'HELLO$'
    EQUALMSG DB 'Strings are equal.$'
    NEQMSG   DB 'Strings are NOT equal.$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX          ; ES must point to data segment for CMPSB

    LEA SI, STRING1     ; Source string
    LEA DI, STRING2     ; Destination string
    MOV CX, LEN1        ; Number of characters to compare
    CLD                 ; Left-to-right scan

    REPE CMPSB         ; Repeat while bytes are equal and CX != 0
    JNE NEQUAL          ; If Zero Flag = 0 (mismatch occurred), jump to
NEQUAL

    LEA DX, EQUALMSG
    JMP SHOW

NEQUAL:
    LEA DX, NEQMSG

SHOW:
    MOV AH, 09H         ; Display string in DX
    INT 21H

    MOV AH, 4CH
```

```
    INT 21H
MAIN ENDP
END MAIN
```

Program 3.3: Check if a String is a Palindrome (`PALIN.ASM`)

- **Concept**: Two pointers — `SI` starts at character 0, `DI` starts at the last character (`LEN - 1`). Compare `[SI]` and `[DI]`, then increment `SI` and decrement `DI`. Loop `LEN / 2` times.

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'MADAM$'
    LEN  EQU $ - STR1 - 1
    YES  DB 'Palindrome.$'
    NO   DB 'Not a palindrome.$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX

    LEA SI, STR1              ; Front pointer
    LEA DI, STR1 + LEN - 1    ; Back pointer
    MOV CX, LEN / 2           ; Check half the string length
    CLD

CHECK:
    MOV AL, [SI]
    CMP AL, ES:[DI]
    JNE DIFFERENT             ; Mismatch found -> Not palindrome
    INC SI                    ; Move forward
    DEC DI                    ; Move backward
    LOOP CHECK                ; Decrements CX, jumps to CHECK if CX != 0

    LEA DX, YES               ; All characters matched
    JMP SHOW

DIFFERENT:
    LEA DX, NO

SHOW:
    MOV AH, 09H
    INT 21H
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Program 3.4: Reverse a String (`REVERSE.ASM`)

- **Concept**: Read from the end of `STR1` (`SI = STR1 + LEN - 1`) and write sequentially to the start of `REV` (`DI = REV`).

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'HELLO$'
    LEN  EQU $ - STR1 - 1
    REV  DB LEN DUP (?)       ; Reserve LEN bytes for reversed output
         DB '$'               ; Terminator for DOS display

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1 + LEN - 1    ; Point to last character of source
    LEA DI, REV               ; Point to first byte of destination
    MOV CX, LEN               ; Counter = number of characters
    CLD

COPY:
    MOV AL, [SI]              ; Fetch character from end
    MOV [DI], AL              ; Write to destination
    DEC SI                    ; Move backwards in source
    INC DI                    ; Move forwards in destination
    LOOP COPY

    LEA DX, REV               ; Print reversed string
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Experiment 4: Booth's Multiplication Algorithm

Concept & Algorithm

Booth's Algorithm multiplies signed binary integers in two's complement form.

- Registers used:
  - M = Multiplicand

    - Q = Multiplier
    - A = Accumulator (initialized to 0)
    - Q_{-1} = 1-bit register (initialized to 0)
    - Count = Number of bits (n)
- Look at the pair (Q_0, Q_{-1}):
    - `1 0`: $A \leftarrow A - M$
    - `0 1`: $A \leftarrow A + M$
    - `0 0` or `1 1`: No arithmetic operation
- **Arithmetic Right Shift (ASR)** of $[A, Q, Q_{-1}]$:
    - Sign bit of A is preserved ($A_{n-1} \rightarrow A_{n-1}$).
    - $A_0 \rightarrow Q_{n-1}$, $Q_0 \rightarrow Q_{-1}$.

Code (`BoothAlgorithm.java`)

```java
import java.util.Scanner;
public class BoothAlgorithm {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the multiplicand: ");
        int M = in.nextInt(); // M: multiplicand.
        System.out.print("Enter the multiplier: ");
        int Q = in.nextInt(); // Q: multiplier.
        booth(M, Q);
        in.close();
    }

    // Q0 and Q-1 decide whether to add or subtract M.
    static void booth(int M, int Q) {
        int A = 0; // Accumulator A.
        int Q_1 = 0; // Previous bit of Q, initially 0.
        int count = 32; // Number of bits.
        while (count-- > 0) {
            int Q0 = Q & 1; // Read the current least-significant bit of Q.
            if (Q0 == 1 && Q_1 == 0) {
                A -= M; // Pair 10: A = A - M.
            } else if (Q0 == 0 && Q_1 == 1) {
                A += M; // Pair 01: A = A + M.
            }

            Q_1 = Q0; // Save Q0 as the next Q-1.
            Q = (Q >>> 1) | ((A & 1) << 31); // Shift the combined A-Q
register.
            A >>= 1; // Arithmetic shift keeps the sign of A.
        }

        System.out.println("Product: " + Q); // Q contains the 32-bit
result.
    }
}
```

Experiment 5: Division Algorithms

Division calculates: $\text{Dividend } (Q) \div \text{Divisor } (M) \rightarrow \text{Quotient } (Q), \text{Remainder } (A)$.

5.1: Restoring Division Algorithm (`RestoringDivision.java`)

Logic:

1. Initialize $A = 0, \text{Count} = n$.
2. Shift Left $[A, Q]$.
3. Subtract divisor: $A = A - M$.
4. Check sign of A:

- If $A < 0$ (most significant bit is 1): Subtraction failed. Set $Q_0 = 0$ and **restore** $A = A + M$.
- If $A \geq 0$: Set $Q_0 = 1$.

5. Decrement count until 0.

```java
import java.util.Scanner;

public class RestoringDivision {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter Dividend (Q): ");
        int Q = in.nextInt(); // Dividend.
        System.out.print("Enter Divisor (M): ");
        int M = in.nextInt(); // Divisor.
        in.close();

        int A = 0; // Partial remainder.
        int count = 32; // Number of bits.
        while (count-- > 0) {
            // Shift the next dividend bit into the partial remainder.
            A = (A << 1) | ((Q >>> 31) & 1);
            Q <<= 1;
            A -= M;
            if (A < 0) {
                A += M; // Restore A after an unsuccessful subtraction.
            } else {
                Q |= 1; // Successful subtraction sets Q0 to 1.
            }
        }

        System.out.println("Quotient (Q): " + Q);
        System.out.println("Remainder (A): " + A);
    }
}
```

5.2: Non-Restoring Division Algorithm (`NonRestoringDivision.java`)

Logic:

Avoids the restorative addition step in the loop:

1. Initialize $A = 0$, $\text{Count} = n$.
2. In each cycle:
   - If $A \geq 0$: Shift Left $[A, Q]$, then $A = A - M$.
   - If $A < 0$: Shift Left $[A, Q]$, then $A = A + M$.
3. Check sign of new A:
   - If $A \geq 0 \implies Q_0 = 1$.
   - If $A < 0 \implies Q_0 = 0$.
4. **Final step (after loop ends)**: If $A < 0$, do a one-time correction: $A = A + M$.

```java
import java.util.Scanner;

public class NonRestoringDivision {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter Dividend (Q): ");
        int Q = in.nextInt(); // Dividend.
        System.out.print("Enter Divisor (M): ");
        int M = in.nextInt(); // Divisor.
        in.close();

        int A = 0; // Partial remainder.
        int count = 32; // Number of bits.
        while (count-- > 0) {
            // Shift the next dividend bit into A.
            A = (A << 1) | ((Q >>> 31) & 1);
            Q <<= 1;

            // Subtract or add the divisor based on the sign of A.
            if (A >= 0) {
                A -= M; // A = A - M.
            } else {
                A += M; // A = A + M.
            }

            // A >= 0 means Q0 is 1; otherwise Q0 is 0.
            if (A >= 0) {
                Q |= 1;
            } else {
                Q &= ~1;
            }
        }

        if (A < 0) {
            A += M; // Final correction if A is negative.
        }
        System.out.println("Quotient (Q): " + Q);
        System.out.println("Remainder (A): " + A);
    }
}
```

Experiment 6: Amdahl's Law

Formulas

Amdahl's Law predicts theoretical speedup when using multiple processors:

$$\text{Speedup } (S) = \frac{1}{(1 - P) + \frac{P}{N}}$$

$$\text{Max Speedup } (N \to \infty) = \frac{1}{1-P}$$

$$\text{Efficiency } (E) = \frac{S}{N}$$

Where:

- P = Parallel fraction of the program ($0 \leq P \leq 1$).
- $(1-P)$ = Serial fraction.
- N = Number of processors.

Code (`AmdahlLaw.java`)

```
public class AmdahlLaw {
    // Speedup = 1 / (serial part + parallel part / processors).
    public static double speedup(double parallelFraction, int processors) {
        return 1.0 / ((1.0 - parallelFraction) + parallelFraction /
processors);
    }

    public static void main(String[] args) {
        double[] parallelFractions = {0.5, 0.75, 0.9, 0.95, 0.99};
        int[] processors = {1, 4, 9, 16}; // Same values used by the
notebook.

        System.out.println("Amdahl's Law (plotting omitted)");
        for (double fraction : parallelFractions) {
            System.out.printf("%nParallel fraction P = %.2f, maximum
speedup = %.4f%n",
                    fraction, 1.0 / (1.0 - fraction));
            for (int processorCount : processors) {
                double speedup = speedup(fraction, processorCount);
                System.out.printf("Processors: %d, Speedup: %.4f,
Efficiency: %.4f%n",
                        processorCount, speedup, speedup / processorCount);
            }
        }
    }
}
```

Experiment 7: Page Replacement Algorithms

Given reference string: `7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1` with `3` page frames.

1. FIFO (First-In, First-Out)

- Replaces the page that was brought in earliest (maintained via a circular pointer: `pointer = (pointer + 1) % FRAME_COUNT`).

2. LRU (Least Recently Used)

- Replaces the page that has not been referenced for the longest period of time (tracked by recording access cycle/timestamp `lastUsed[frame] = i`).

Code (`PageReplacement.java`)

```
public class PageReplacement {
    // Reference string and number of available page frames.
    static final int[] REFERENCES = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3,
            0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    static final int FRAME_COUNT = 3;

    static void fifo() {
        int[] frames = {-1, -1, -1};
        int pointer = 0, hits = 0, faults = 0;
        for (int i = 0; i < REFERENCES.length; i++) {
            boolean hit = contains(frames, REFERENCES[i]); // Page already
loaded?
            if (hit) {
                hits++;
            } else {
                frames[pointer] = REFERENCES[i];
                pointer = (pointer + 1) % FRAME_COUNT;
                faults++;
            }
            System.out.printf("%d %s%n", i, hit ? "H" : "F");
        }
        printTotals("FIFO", faults, hits);
    }

    static void lru() {
        int[] frames = {-1, -1, -1};
        int[] lastUsed = {-1, -1, -1};
        int hits = 0, faults = 0;
        for (int i = 0; i < REFERENCES.length; i++) {
            int page = REFERENCES[i];
            int index = indexOf(frames, page);
            boolean hit = index >= 0;
            if (hit) {
                lastUsed[index] = i;
                hits++;
            } else {
                // Replace the page whose last-use position is smallest.
                index = indexOf(lastUsed, minimum(lastUsed));
                frames[index] = page;
                lastUsed[index] = i;
                faults++;
            }
```

```
                System.out.printf("%d %s%n", i, hit ? "H" : "F");
            }
            printTotals("LRU", faults, hits);
        }

        static boolean contains(int[] values, int target) {
            return indexOf(values, target) >= 0;
        }

        static int indexOf(int[] values, int target) {
            for (int i = 0; i < values.length; i++) {
                if (values[i] == target) return i;
            }
            return -1;
        }

        static int minimum(int[] values) {
            int result = values[0];
            for (int value : values) result = Math.min(result, value);
            return result;
        }

        static void printTotals(String policy, int faults, int hits) {
            System.out.printf("%n%s: Faults = %d, Hits = %d, Fault/Hit ratio =
    %.2f%n",
                    policy, faults, hits, (double) faults / hits);
        }

        public static void main(String[] args) {
            System.out.println("FIFO Page Replacement");
            fifo();
            System.out.println("\nLRU Page Replacement");
            lru();
        }
    }
```

Experiment 8: Cache Direct Mapping

Architecture & Bit Calculations

In a direct-mapped cache:

- $\text{No. of Main Memory Blocks} = \frac{\text{Main Memory Size}}{\text{Block Size}}$
- $\text{No. of Cache Lines} = \frac{\text{Cache Memory Size}}{\text{Block Size}}$
- $\text{Block Index Bits } (BI) = \log_2(\text{Blocks})$
- $\text{Cache Line Index Bits } (CI) = \log_2(\text{Lines})$
- $\text{Tag Bits } (TI) = BI - CI$

Mapping Formula

For every Main Memory Block Index (BI):

- Cache Index $(CI) = BI \pmod{\text{No. of Lines}}$
- Tag Index $(TI) = \lfloor BI / \text{No. of Lines} \rfloor$

Code (DirectMapping.java)

```
import java.util.Scanner;

public class DirectMapping {
    // Print a number using exactly width bits.
    static String binary(int value, int width) {
        if (width == 0) {
            return "-";
        }
        return String.format("%" + width + "s",
Integer.toBinaryString(value))
                .replace(' ', '0');
    }

    static int log2(int value) {
        return Integer.numberOfTrailingZeros(value);
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size of main memory in bytes: ");
        int mainMemory = in.nextInt();
        System.out.print("Enter the size of cache memory in bytes: ");
        int cacheMemory = in.nextInt();
        System.out.print("Enter the block size in bytes: ");
        int blockSize = in.nextInt();
        in.close();

        int blocks = mainMemory / blockSize; // Main-memory block count.
        int lines = cacheMemory / blockSize; // Cache-line count.
        int biBits = log2(blocks);
        int ciBits = log2(lines);
        int tagBits = biBits - ciBits;

        System.out.println("\nNo. of blocks in main memory: " + blocks);
        System.out.println("No. of Lines in cache memory: " + lines);
        System.out.printf("Bit Widths -> BI: %d bits | CI: %d bits | TI: %d
bits%n%n",
                biBits, ciBits, tagBits);
        System.out.println("| Block Index (BI) | Cache Index (CI) | Tag
Index (TI) |");
        System.out.println("|------------------|------------------|--------
--------|");
        for (int block = 0; block < blocks; block++) {
            int cacheLine = block % lines; // BI modulo number of lines.
            int tag = block / lines; // Remaining upper bits.
```

```
            System.out.printf("| %-16s | %-16s | %-14s |%n",
                    binary(block, biBits), binary(cacheLine, ciBits),
binary(tag, tagBits));
        }
    }
}
```

Experiment 10: Pipelining Simulation

Theory & Formulas

A standard 5-stage RISC processor pipeline:

1. **IF**: Instruction Fetch
2. **ID**: Instruction Decode / Register Fetch
3. **EX**: Execute
4. **MEM**: Memory Access
5. **WB**: Write Back

Equations

- **Non-pipelined execution time**: $T_{np} = n \times k$ clock cycles (where n = instructions, k = stages = 5).
- **Ideal pipelined execution time**: $T_p = k + (n - 1)$ cycles.
- **Actual execution time with stalls**: $T_{\text{actual}} = T_p + \text{stalls}$.
- **Speedup**:

$$S_{\text{ideal}} = \frac{T_{np}}{T_p}, \quad S_{\text{actual}} = \frac{T_{np}}{T_{\text{actual}}}$$

- **Efficiency**:

$$\eta = \frac{S}{k} \times 100\%$$

Hazard in this simulation:

- I_1: `ADD R1, R2, R3` (writes result into R_1 in stage WB, cycle 5).
- I_2: `SUB R4, R1, R5` (needs value of R_1 in stage EX).
- Result: I_2 must **stall for 2 cycles** in EX until I_1 finishes WB.

Code (`pipelining.java`)

```
import java.util.Scanner;

public class pipelining {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```
        // 1. Take User Input
        System.out.print("Number of Instructions (n) = ");
        int n = scanner.nextInt();

        System.out.print("Pipeline Stages (k) = ");
        int k = scanner.nextInt();

        // 2. Define the sample instructions (hardcoded for simple
simulation)
        String i1 = "ADD R1,R2,R3";
        String i2 = "SUB R4,R1,R5";
        String i3 = "AND R6,R7,R8";

        System.out.println("\n5-STAGE PIPELINE SIMULATION\n");
        System.out.printf("%-7s %-15s %-15s %-15s %-15s %-15s%n", "Cycle",
"IF", "ID", "EX", "MEM", "WB");

        // 3. Print Cycle-by-Cycle Table
        for (int cycle = 1; cycle <= 10; cycle++) {
            String ifStage = "--", idStage = "--", exStage = "--", memStage
= "--", wbStage = "--";

            switch (cycle) {
                case 1: ifStage = i1; break;
                case 2: ifStage = i2; idStage = i1; break;
                case 3: ifStage = i3; idStage = i2; exStage = i1; break;
                case 4: ifStage = i3; idStage = i2; exStage = "STALL";
memStage = i1; break;
                case 5: ifStage = i3; idStage = i2; exStage = "STALL";
memStage = "STALL"; wbStage = i1; break;
                case 6: idStage = i3; exStage = i2; memStage = "STALL";
wbStage = "STALL"; break;
                case 7: exStage = i3; memStage = i2; wbStage = "STALL";
break;
                case 8: memStage = i3; wbStage = i2; break;
                case 9: wbStage = i3; break;
                case 10: break;
            }

            System.out.printf("%-7d %-15s %-15s %-15s %-15s %-15s%n",
cycle, ifStage, idStage, exStage, memStage, wbStage);
        }

        // 4. Calculations (Lab Manual Formulas)
        int totalStalls = 2;
        int nonPipelinedCycles = n * k;                      // T_np = n *
k
        int idealPipelinedCycles = k + (n - 1);             // T_p = k + (n
- 1)
        int actualPipelinedCycles = idealPipelinedCycles + totalStalls;

        // Theoretical (Ideal) Metrics
        float idealSpeedup = (float) nonPipelinedCycles /
idealPipelinedCycles;
```

```
        float idealEfficiency = idealSpeedup / k;

        // Actual (With Hazard Stalls) Metrics
        float actualSpeedup = (float) nonPipelinedCycles /
actualPipelinedCycles;
        float actualEfficiency = actualSpeedup / k;

        // 5. Display Summary Results
        System.out.println("\n--- SIMULATION SUMMARY ---");
        System.out.println("Total Instructions: " + n);
        System.out.println("Total Stall Cycles: " + totalStalls);
        System.out.println("Ideal Cycles: " + idealPipelinedCycles);
        System.out.println("Extra Cycles: " + totalStalls);

        System.out.println("\n--- PERFORMANCE METRICS (IDEAL) ---");
        System.out.println("Non-Pipelined Cycles = " + nonPipelinedCycles);
        System.out.println("Pipelined Cycles = " + idealPipelinedCycles);
        System.out.printf("Speedup = %.2f%n", idealSpeedup);
        System.out.printf("Efficiency = %.2f (%.1f%%)%n", idealEfficiency,
idealEfficiency * 100);

        System.out.println("\n--- PERFORMANCE METRICS (ACTUAL WITH STALLS)
---");
        System.out.println("Actual Pipelined Cycles = " +
actualPipelinedCycles);
        System.out.printf("Actual Speedup = %.2f%n", actualSpeedup);
        System.out.printf("Actual Efficiency = %.2f (%.1f%%)%n",
actualEfficiency, actualEfficiency * 100);

        scanner.close();
    }
}
```

Quick Cheat-Sheet for the Exam

8086 Programs

Exp	Remember this order	One-line recall
1.1 Hello World	Set `DS` -> load message -> `AH = 09H` -> `INT 21H`	DOS prints until `$`; exit with `AH = 4CH`.
1.2 Addition	Load `NUM1` -> `ADD NUM2` -> store `SUM`	`DW` = 16-bit word; use `AX` for the word result.
2 Block Transfer	Set `DS:SI`, `ES:DI`, `CX` -> `CLD` -> `REP MOVSW`	`MOVSW` copies one word; `CX` is the number of words.
3.1 String Length	`SI` at start -> clear `CX` -> `LODSB` -> test `$` -> increment `CX`	`CX` counts characters before `$`.

Exp	Remember this order	One-line recall
3.2 Compare Strings	Set `DS` and `ES` -> set `SI`, `DI`, `CX` -> `REPE CMPSB`	`JNE` means a mismatch was found.
3.3 Palindrome	`SI` at start, `DI` at end -> compare -> move inward	Only `LEN / 2` pairs need checking.
3.4 Reverse String	`SI` at end, `DI` at output start -> copy backward to forward	Decrease `SI`; increase `DI`.

Algorithms and Performance

Exp	Algorithm	Must-remember rule
4 Booth	Signed multiplication	`(Q0, Q-1)`: `10` -> `A - M`; `01` -> `A + M`; then arithmetic right shift.
5.1 Restoring division	Shift, subtract, restore if negative	Negative `A` -> `Q0 = 0`, then `A = A + M`; otherwise `Q0 = 1`.
5.2 Non-restoring division	Add or subtract based on the previous sign	`A >= 0` -> subtract `M`; `A < 0` -> add `M`; correct negative `A` at the end.
6 Amdahl's Law	Speedup and efficiency	$S = \frac{1}{(1-P)+P/N}$; maximum speedup $= \frac{1}{1-P}$; $E = S/N$.
7 FIFO	Oldest loaded page is replaced	Advance the circular pointer after every page fault.
7 LRU	Least recently used page is replaced	Replace the frame with the smallest `lastUsed` value.
8 Direct mapping	Main-memory block -> cache line	`CI = BI % lines`; `TI = BI / lines`; `BI bits = CI bits + TI bits`.
10 Pipelining	IF -> ID -> EX -> MEM -> WB	$T_{np} = n \times k$; ideal $T_p = k + n - 1$; actual time adds stall cycles.

Last-Minute Reminders

- **8086 string direction:** `CLD` means forward; `STD` would mean backward.
- **8086 string registers:** source is `DS:SI`; destination is `ES:DI`; repetition count is `CX`.
- **DOS services:** `AH = 09H` displays a `$`-terminated string; `AH = 4CH` exits.
- **Division notation:** `Q` starts as the dividend and becomes the quotient; `A` is the partial remainder.
- **Pipeline metrics:** `Speedup = T_{np}/T_p`; `Efficiency = Speedup/k`.